

Why Observational AI-Agent Pull Request Datasets Cannot Support Behavioral Inference: A Validity Analysis of AIDev

Ahmed Mursal
School of Computing
Edinburgh Napier University, UK
40646515@live.napier.ac.uk

Abstract—We examine the AIDev dataset’s suitability for AI agent behavioral research and identify fundamental validity failures that preclude meaningful analysis. Agent attribution shows obvious errors (GitHub Copilot: 379 users vs. millions in reality), observations violate statistical independence assumptions, and test detection exhibits systematic bias. We demonstrate why observational pull request datasets cannot support behavioral inferences about AI tools without controlled experimental designs. This methodology critique provides guidelines for valid AI tool research and warns against common pitfalls in large-scale observational studies of software development tools.

I. INTRODUCTION

AI coding assistants are increasingly studied through observational analysis of development artifacts. The AIDev dataset represents a prominent example, containing 932,791 pull requests with agent attribution labels. However, such observational approaches face fundamental methodological challenges that compromise validity.

We examine whether pull request datasets can support reliable behavioral inferences about AI tools. Our analysis reveals critical failures in: (1) agent attribution reliability, (2) statistical assumption compliance, and (3) measurement validity. These issues collectively preclude behavioral inference from observational PR data.

Contribution. We provide a methodological framework for evaluating AI tool datasets and demonstrate why controlled experimental designs are necessary for valid behavioral research. This critique aims to improve research practices in software engineering tool evaluation.

II. METHODOLOGY FAILURES IN OBSERVATIONAL AI RESEARCH

A. Agent Attribution Unreliability

The AIDev dataset assigns agent labels through metadata extraction and self-reports. However, examination reveals systematic attribution failures:

Impossible User Counts. GitHub Copilot appears with only 379 users despite millions of actual users worldwide. This indicates either severe under-detection or mislabeling of the most widely-used tool.

Implausible Distributions. Lesser-known tools (Claude Code) show more users than established tools (Copilot), contradicting known adoption patterns.

Single-User Anomalies. Devin appears as 29,744 pull requests from one user, suggesting either data corruption or misattribution of bulk automated changes.

These attribution failures destroy internal validity because behavioral comparisons require accurate tool identification.

B. Statistical Independence Violations

Observational PR datasets violate fundamental assumptions of common statistical tests:

Repository Clustering. Multiple PRs from the same repository share policies, practices, and maintainer preferences. These are not independent observations.

User Clustering. Multiple PRs from the same developer reflect individual coding style and project context rather than tool behavior.

Temporal Clustering. PRs within projects often follow sequential development phases that introduce dependencies.

Standard chi-square tests assume independence. Applying them to clustered observations produces invalid p-values and inflated effect sizes.

C. Measurement Validity Problems

Test detection through keyword matching exhibits systematic bias:

Language Bias. Detection favors Python/JavaScript frameworks (pytest, jest) while missing ecosystems like Go, Rust, and C#.

False Positives. Phrases like “tested in production” or “integration issues” trigger detection without indicating actual tests.

Context Confusion. Mixing file paths, commit messages, and PR descriptions creates inconsistent detection criteria.

Validation Inadequacy. Manual validation of 200 samples cannot detect systematic biases across 932k observations spanning multiple languages and domains.

III. WHY BEHAVIORAL INFERENCE FAILS

Even with perfect attribution and measurement, observational PR data cannot support behavioral inference due to confounding:

A. Selection Effects

Developers choose tools based on:

- Project requirements (testing vs. prototyping)
- Language ecosystems
- Organizational policies
- Experience levels
- Task complexity

These selection factors correlate with testing practices, making tool choice a confounded variable.

B. Usage Context Variation

The same tool used for different purposes produces different artifacts:

- Experimentation vs. production code
- Individual vs. team development
- Open source vs. enterprise projects
- Different programming languages

Observational data cannot distinguish tool effects from context effects.

IV. REQUIREMENTS FOR VALID AI TOOL RESEARCH

A. Controlled Experimental Design

Valid behavioral research requires:

Random Assignment. Participants randomly assigned to tools eliminates selection bias.

Controlled Tasks. Standardized programming tasks enable fair comparison.

Verified Attribution. Direct tool instrumentation ensures accurate measurement.

Appropriate Controls. Baseline conditions without AI assistance provide comparison points.

B. Measurement Best Practices

Language-Aware Detection. Test identification must account for language-specific patterns and frameworks.

Manual Validation. Stratified sampling across languages, projects, and tools with expert review.

Process Metrics. Measure development process, not just final artifacts.

Quality Assessment. Evaluate test effectiveness, not just presence.

C. Statistical Approach

Cluster-Robust Methods. Account for non-independence through mixed-effects models or cluster-robust standard errors.

Matched Sampling. Control for confounders through propensity score matching or stratification.

Effect Size Interpretation. Report practical significance alongside statistical significance.

V. GUIDELINES FOR DATASET EVALUATION

Before using observational datasets for behavioral research:

A. Attribution Validation

- 1) Verify user counts match known adoption patterns
- 2) Check for impossible or extreme distributions
- 3) Investigate single-user anomalies
- 4) Cross-validate with independent data sources

B. Independence Assessment

- 1) Quantify clustering within repositories and users
- 2) Test assumptions using diagnostic statistics
- 3) Plan appropriate statistical approaches for clustered data

C. Measurement Validation

- 1) Stratify validation across relevant dimensions
- 2) Test for systematic bias in detection methods
- 3) Validate against expert judgment on representative samples

VI. IMPLICATIONS FOR SOFTWARE ENGINEERING RESEARCH

Observational Studies Have Limits. Large samples cannot overcome fundamental validity threats. Size does not equal validity.

Tool Research Needs Experiments. Behavioral claims about AI tools require controlled experimental validation, not observational association studies.

Dataset Quality Matters. Researchers must validate attribution accuracy and measurement validity before analysis.

Statistical Methods Must Match Data Structure. Clustered observations require cluster-aware statistical approaches.

VII. CONCLUSION

The AIDev dataset, despite its impressive scale, cannot support valid behavioral inferences about AI coding tools due to unreliable attribution, independence violations, and measurement bias. This analysis demonstrates why observational pull request datasets are unsuitable for tool behavioral research without extensive validation and appropriate statistical methods.

Future research should prioritize controlled experimental designs with verified tool attribution. The software engineering community must resist the temptation to draw behavioral conclusions from large but fundamentally flawed observational datasets.

ACKNOWLEDGMENTS

We thank H. Li et al. for releasing AIDev, which enabled this methodological analysis. We hope this critique contributes to improved research practices in software engineering tool evaluation.

REFERENCES

REFERENCES

- [1] H. Li et al., "The Rise of AI Teammates in Software Engineering (SE 3.0)," arXiv:2409.18925, 2024.
- [2] M. Chen et al., "Evaluating Large Language Models Trained on Code," arXiv:2107.03374, 2021.
- [3] A. Bareiß et al., "Code generation tools in practice: A user study with GitHub Copilot," in Proc. MSR, pp. 285-297, 2023.